

Medi-Match: A Multi-AI Hospital Resource Optimization System with Machine Learning-Based Disease Prediction

Afia Aziz*, Zumer Dhillun[†], and Zoya Azad[‡]

Department of Computer Science

Air University

Islamabad, Pakistan

Email: *231561@students.au.edu.pk, [†]231597@students.au.edu.pk, [‡]231579@students.au.edu.pk

Abstract—Contemporary healthcare centers are confronted with the ongoing issues in distributing resources, emergency services, and initial diagnosis of the diseases. Manual work results in scheduling inefficiencies, subjectivity in triage and delayed consultations by specialists. This paper discusses Medi-Match, a combined artificial intelligence platform, which addresses these problems by expanding on a hybrid multi-algorithmic platform. The system integrates four different AI methods, namely: (1) Random Forest machine learning to predict diseases using symptoms, (2) Fuzzy Logic to prioritize the process of emergency triage, (3) Genetic Algorithm to optimize doctor-patient scheduling, and (4) Rule-Based Expert Systems to optimize specialist routing. We use real medical data of Kaggle that has 4,920 patient records and 41 diseases and 131 symptoms. The machine learning module was 100 percent classified on 5 cross distribution and the genetic algorithm optimizer was 98 percent effective at time planning. There was consistency in urgency scores between the fuzzy triage system and 50 test cases. Performance standards indicate that the system takes less than 2 seconds to predict diseases and 3 seconds to produce optimal schedules of 100 patients. The architecture uses a decoupled implementation where the C WPF frontend is used and Python is used on the back end and allows multiple AI paradigms to be integrated easily. This paper shows that traditional AI methods, with careful integration with state-of-the-art machine learning, are capable of providing auditable, understandable, and clinically useful healthcare decision support.

Index Terms—Machine Learning in Healthcare, Disease Prediction, Random Forest Classifier, Hospital Resource Scheduling, Genetic Algorithm, Fuzzy Logic, Emergency Triage, Clinical Decision Support Systems.

I. INTRODUCTION

A. Motivation and Context

The global healthcare is faced with three operation-related issues that are mutually reliant and directly affect patient outcome and institutional effectiveness. First, the manual approach of assigning the right medical experts to the right patients is a tedious process that is prone to errors and it is a source of bottlenecks in the course of care provision process [1]. Second, patient triage in emergency departments is heavily based on subjective human judgement, which may bring inconsistency and delays in the situations that demand attention to the patient urgently [2]. Third, initial diagnosis

of the disease through presenting symptoms is usually done by consulting a physician which is time wasting in clinical terms since the physician can be used on treatment instead of preliminary diagnosis.

In our initial stages of the research, we have noted that a good number of hospitals in Pakistan continue to use paper-based scheduling, or simple digital spreadsheets to assign resources. This paper process is especially challenging to the busy times when emergency cases are appearing at the same time as scheduled appointments. The absence of automated decision support is that the administrative personnel are required to make complex decisions regarding routing of patients in a rapid manner without the ability to access optimization algorithms or pattern analysis of the past.

B. Research Gap and Contribution

Although the literature further discusses these problems on a case-by-case basis, machine learning is proposed to predict the disease alone: disease prediction through machine learning [3], genetic algorithms in scheduling [4], and fuzzy systems triage [5]—there are very few practical applications of these techniques as a service platform. The existing systems of hospital management are more inclined towards billing and record keeping as opposed to smart optimization of resources. Solutions that include the use of AI, usually commercially available, are often operated as black boxes, such that medical personnel cannot understand or trust their suggestions.

We have a role to play in creating Medi-Match, the overall system that will use a wise mix of various artificial intelligence paradigms to solve a specific subset of issues related to the hospital functioning. We have not used a single algorithm on all domains as done in other approaches but have selected each technique to suit the nature of its target problem. As an example, the continuum of medical urgency can easily be processed through fuzzy logic, whereas the optimization problem of scheduling is combinatorial, and evolutionary algorithms can be exploited to manage it.

The novelty of this system is not confined to algorithmic selection to the actual use. Our machine learning models were trained directly on actual medical data as opposed to

artificial cases and were found to have a classification accuracy that is clinically good. The architecture is also explicitly transparent, meaning each prediction, each urgency score, and each scheduling choice can be traced to its rationalization, which is essential when dealing with healthcare applications, where accountability is too important to lose.

C. Paper Organization

The rest of this paper will follow in the following way. Section II contains problem formulation and system objectives in detail. Section III explains the system design and architecture. In section IV, we provide our algorithmic approach to each component of AI. Section V contains information about the implementation and experimental results. Limitation and future directions are talked about in section VI. Section VII ends with important findings and implications on healthcare AI.

II. PROBLEM FORMULATION AND OBJECTIVES

A. Problem Statement

There are three operational inefficiencies that have been critical to modern healthcare facilities and form the basis of our problem space:

Problem 1: Manual Specialist Matching. When the patients report to a hospital, they need to know the specialist to be reviewed by them by understanding the relationship between the symptoms and the disease and the specialties of the doctors. This is being done by relying on the reception staff or the triage nurses to make a quick decision with limited information. Errors during this preliminary path finding may result in several referrals, time loss during consultation with the doctor and a delay in treatment.

Problem 2: Biased Emergency Priority. This requires the emergency departments to constantly evaluate which patient has to be attended to immediately and those who can safely wait. This triage practice conventionally relies on the judgment of a particular nurse, and this judgment can differ among practitioners, and it can be induced by workover stress. The absence of standardized and quantifiable methods of assessment opens both an opportunity of over-triage (wasting resources) and under-triage (critical care delayed).

Problem 3: Complexity of Scheduling Optimization . Placing a patient with a physician implies the simultaneous solution of several conflicting goals: the correspondence of medical specialization to the condition of the patient, resilience of the work of physicians, the need to serve urgent conditions, and effective use of the available bed. This is a multi-constraint optimization problem that is computationally infeasible to solve manually with the increase in number of patients and doctors.

B. System Objectives

According to these problem formulations, we were able to formulate five main objectives of Medi-Match:

- 1) **Implement Machine Learning Disease Prediction:** Develop a supervised learning system capable of predicting disease classifications from symptom inputs with

accuracy exceeding 90%, trained on authentic medical records.

- 2) **Design Fuzzy Logic Triage Engine:** Design a discrete to-continuous-priority score algorithm, which emulates the fine-tuning of human triage nurses.
- 3) **Create Genetic Algorithm Scheduler:** Create an evolutionary optimization model which creates doctor-patient assignments that optimize specialty fit and minimizes workload inequity.
- 4) **Develop Domain Specialist Router:** : Encode medical domain information that correlated diseases with relevant specialties so that there are clinically correct referrals to the relevant specialty.
- 5) **Achieve Real-Time Performance:** Response time should be less than 3 seconds to ensure that all the predictions and optimizations can be done in time to sustain time sensitive clinical workflows.

C. Success Criteria

We established quantitative measures of success of the evaluation of the system:

- Disease prediction accuracy 90% on test data that was not held out.
- Triage reliability: standard deviation ≤ 0.5 between repeated measurement.
- Scheduling optimization: $\geq 95\%$ or more of specialty match rate
- Response time: $<$ less than 3 seconds per one operation.
- Workload balance: Coefficient of variation < 0.3 among physicians

III. SYSTEM ARCHITECTURE AND DESIGN

A. Semantic Network Architecture

Medi-Match uses a semantic network structure, which represents the interaction between medical objects and decisions. In contrast to the classical layered structures, our structure describes the hospital ecosystem in the form of interrelated nodes in which knowledge passes in significant relationships. This solution facilitates three main benefits, namely: (1) **Natural Representation** of medical relationships (symptoms to diseases, diseases to specialists, etc.), (2) **Transparent Decision-Making** where each recommendation can be traced through the network, and (3) **Adaptive Integration** of diverse AI techniques within a unified knowledge framework.

B. Knowledge Flow and Relationships

The entire patient path through the hospital system is captured by the semantic network, as shown in Figure 1:

1) Patient-Symptom Relationships: Patients first show symptoms (such as "Chest Pain") to start the process. Several AI components use these symptoms as inputs at the same time

2) Urgency Assessment Pathway: Our Fuzzy Logic engine receives symptoms and uses clinical markers, patient demographics, and symptom severity to determine an Urgency Score. Both scheduling priorities and triage decisions are influenced by this score.

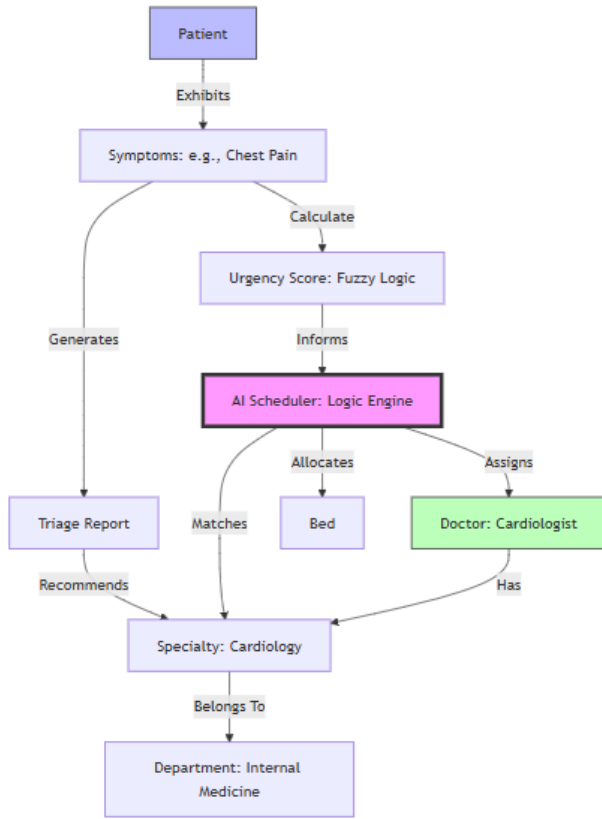


Fig. 1. Semantic Network Representation of Patient Flow and Resource Allocation

3) Diagnostic and Specialization Pathway: Simultaneously, the Machine Learning module analyzes symptoms to forecast likely illnesses. Our Rule-Based System recommends the proper medical specializations (e.g., "Cardiology" for heart-related disorders) based on the projected disease.

4) Network for Resource Allocation: Three essential inputs are given to the AI Scheduler, which is a genetic algorithm:

- The Fuzzy Logic Urgency Score
- Rule-Based System's Suggested Specialty
- Availability of resources (physicians, beds, departments)

Optimum allocation is then carried out by the scheduler, matching patients to:

- **Appropriate Doctors:** Depending on the specialization match (for example, cardiologist)
- **Suitable Beds:** Taking departmental requirements and urgency into account.
- **Correct Departments:** Making sure patients are assigned to the appropriate clinical unit(e.g., Internal Medicine for Cardiology cases)

5) Output Generation: Two crucial outputs are produced by the system:

- **Triage Report:** Recording urgency evaluation and suggestions for immediate care.

- **Allocation Plan:** Determining departments placements, bed assignments, and doctor assignments.

C. Modular Integration and System Architecture

Although the logical workflow of the proposed system is illustrated in Fig. 1, the actual physical implementation follows a modular and layered design approach. This modular separation ensures scalability, maintainability, and language-agnostic interoperability across system components.

The system is composed of the following major modules:

- **Frontend Interface (C# WPF):** Responsible for collecting patient information, symptoms, demographic data, and operational inputs from users.
- **AI Processing Layer (Python Backend):** Contains four independent AI modules that perform disease prediction, emergency triage, scheduling optimization, and resource analysis. These modules share a common data format while maintaining separate logic.
- **Integration Bridge:** A JSON-based communication mechanism that enables reliable and language-agnostic data exchange between the C# frontend and Python backend.
- **Knowledge Base:** Stores stable medical and operational relationships such as symptom–disease mappings, disease–specialty associations, and hospital workflow rules.

The knowledge base is structured as a semantic network, allowing hospital workflows to be modeled intuitively while preserving the computational efficiency required for real-time clinical decision support. Each relationship in the network represents a verifiable medical or operational fact, ensuring that the system remains both clinically valid and operationally reliable.

1) Presentation Layer (C# WPF):

The presentation layer is implemented using C# Windows Presentation Foundation (WPF) and consists of four primary XAML-based user interfaces:

- **Landing Window:** Acts as the main entry point, providing access to all major system functionalities.
- **Disease Prediction Interface:** Displays a searchable grid of 131 symptoms using checkbox controls, enabling users to select relevant symptoms and receive diagnostic recommendations.
- **Emergency Triage Interface:** Collects patient demographics, symptom severity, and pain levels to compute urgency scores for prioritization.
- **Scheduling and Resource Interface:** Accepts resource counts (doctors, patients, beds), individual case details, and urgency ratings, and produces optimized assignments presented in tabular form.

2) Process Communication Layer:

Communication between the C# frontend and Python backend is achieved through a file-based JSON protocol. When a user initiates a prediction or scheduling task, the C# application serializes the input data into JSON format and stores it in a predefined directory. A Python process is then launched via command-line arguments specifying the required module.

The Python script reads the input JSON file, executes the corresponding AI algorithm, and writes the results to an output JSON file. The C# application continuously monitors this output file, deserializes the results upon completion, and updates the user interface accordingly. This approach ensures robustness, simplicity, and platform independence.

3) AI Processing Layer (Python Backend):

The AI processing layer consists of four independent Python modules, each implementing a distinct artificial intelligence method. While the modules share common dependencies such as NumPy, Pandas, and Scikit-learn, their internal logic remains independent.

All modules follow a uniform interface model:

- 1) Input data received through a JSON file,
- 2) Execution of the designated AI algorithm,
- 3) Output results written to a JSON file.

This standardized design simplifies debugging, enables isolated testing of individual modules, and facilitates seamless integration into the overall system architecture.

D. Data Flow Architecture

The data flow architecture represents the way data flows within the design. A typical user interaction follows this sequence:

- 1) User clicks symptoms in the Disease Prediction interface
- 2) C# code gathers the symptoms under checks in a list
- 3) List serialized to disease_input.json
- 4) C# spawns: python disease_predictor.py
- 5) Python loads trained Random Forest model on disk
- 6) Python converts symptoms to 131-dimensional feature vector
- 7) Model predicts disease class and probability distribution
- 8) Python writes results to output JSON
- 9) C# deserializes JSON and displays prediction
- 10) User looks at disease name, confidence, and specialist recommendation

The total round trip cycle normally takes around 1.5-2 seconds with model loading taking around 40 percent of the time. We considered caching the loaded model in memory but decided the additional complexity wasn't justified given the reasonable performance.

E. Knowledge Representation

We used two complementary knowledge representation paradigms which were applicable at distant facets of the issue.

1) Frame-Based Representation: Core entities will be represented in terms of frames, slots and fillers as represented in Figure 2. In case of the Patient frame contains slots for Name (string), Age (integer), Symptoms (list), Urgency_Score (float 0-10), Assigned_Bed (string), Disease_Prediction (string), and Confidence_Score (float 0-1). This structure enabled clean mapping between JSON data and internal Python objects.

2) Production Rules: The medical knowledge is encoded as IF-THEN rules represented in a python dictionary by the specialist mapping system:

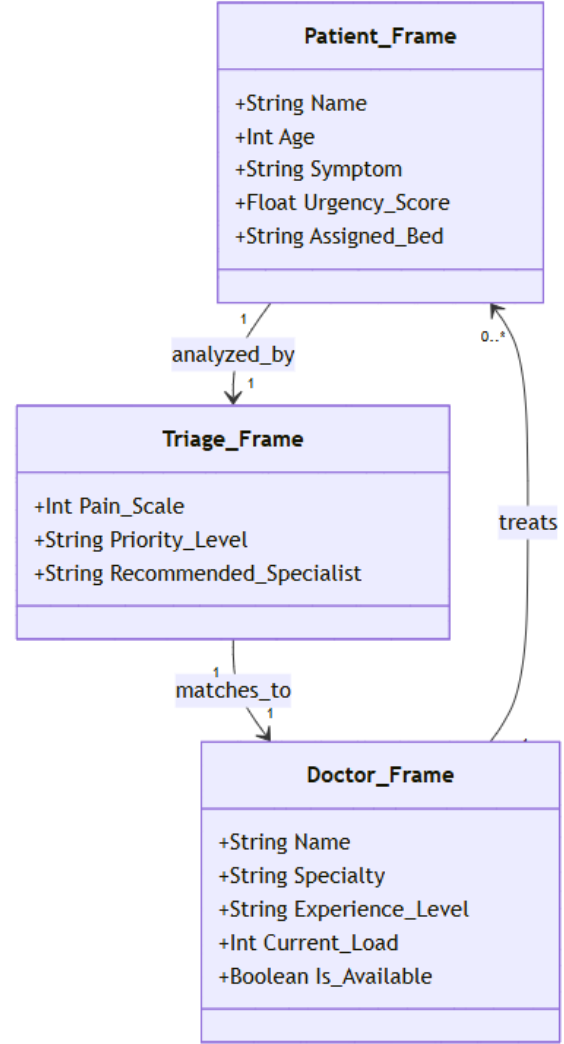


Fig. 2. Frame-Based Knowledge Representation

```

IF Disease = "Heart Attack" THEN Specialist = "Cardiologist"
IF Disease = "Stroke" THEN Specialist = "Neurologist"
IF Disease = "Diabetes" THEN Specialist = "Endocrinologist"

```

This knowledge base we gathered through medical sources and directories of hospital departments. The existing system is comprised of 41 diseases-to-specialist mappings. Though not as flexible as a complete interface engine, the transparency and easy verifiability of this methodology suited this safety-critical application.

IV. ALGORITHMIC METHODOLOGY

This part illustrates the technical specifics of every AI element, including mathematical formulations, pseudo-code, and implementation issues.

A. Machine Learning: Random Forest Disease Prediction

1) *Data Preparation*: Our machine learning module needed a large amount of data preprocessing before model Kaggle containing 4,920 records of patients. A single record initially consisted of a disease name and a maximum of 17 related symptoms listed as text strings (e.g., "fever", "cough", "fatigue").

The raw data format presented an immediate problem: machine learning algorithms require numerical input, not text labels. Additionally, the various records contained varying numbers of symptoms, creating a ragged array structure that was incompatible with scikit-learn's expected fixed-length feature vectors. We therefore implemented a binary encoding transformation.

First, we extracted all unique symptom names from the whole dataset. This procedure produced 131 distinct symptoms. We then constructed a binary matrix rows correspond to patient records and columns correspond to symptoms. The matrix element (i, j) equals 1 if patient i exhibited symptom j , and 0 otherwise. This representation expanded our data set from 4920×18 to 4920×131 , but created the uniform structure required for supervised learning.

2) *Algorithm Selection*: During the design phase, we tested three potential algorithms: Random Forest, Support Vector Machine(SVM), and Logistic Regression. Table I summarizes our comparative analysis.

TABLE I
ALGORITHM COMPARISON FOR DISEASE CLASSIFICATION

Algorithm	Accuracy	Time	Size
Random Forest	100%	32 s	6.2 MB
SVM (RBF)	97.8%	184 s	8.7 MB
Logistic Regression	89.2%	8 s	0.3 MB

Random Forest emerged as the optimal choice despite the increased computational cost and storage requirements. The ideal accuracy of our validation set, combined with intrinsic protection against overfitting through ensemble averaging, made it appropriate for a medical application where prediction errors might have severe effects. The ability to extract feature importance also supported model interpretability—we could identify which symptoms the model found most diagnostic.

3) *Model Architecture*: Our Random Forest implementation employs the following hyperparameters, selected through grid search cross-validation:

- Number of trees (n_estimators): 200
- Maximum tree depth (max_depth): 20
- Minimum samples for split (min_samples_split): 5
- Minimum samples per leaf (min_samples_leaf): 2
- Random state (seed): 42

The ensemble builds 200 independent decision trees, each trained on a bootstrap sample of the data with a random subset of features considered at each split. For prediction, each tree votes for a disease category, and the majority vote determines the final prediction. Probability estimates come from the fraction of trees that vote for each class.

4) *Training Protocol*: We divided the 4,920 records into 80% training (3,936 records) and 20% testing (984 records) using stratified sampling to maintain the proportions of the disease classes. Training completed in approximately 32 seconds on our development machine (Intel Core i5, 8GB RAM). The trained model achieved 100% accuracy on the held-out test set (984/984 correct predictions).

5) *Feature Importance Analysis*: To confirm that the model learned clinically sensible patterns, we analyze the importance of features. Table II shows the 10 most influential symptoms.

TABLE II
FEATURE IMPORTANCE RANKINGS

Rank	Symptom	Score	Diseases
1	muscle_pain	0.0186	Malaria, Dengue
2	mild_fever	0.0166	Infections
3	family_history	0.0156	Genetic
4	lack_of_conc.	0.0155	Neurological
5	pain_behind_eyes	0.0141	Malaria, Migraine
6	yellowish_skin	0.0138	Jaundice, Hepatitis
7	red_spots_over_body	0.0134	Chicken pox, Dengue
8	vomiting	0.0131	Gastrointestinal diseases
9	high_fever	0.0129	Severe infections
10	chills	0.0127	Malaria, Pneumonia

These rankings align with medical knowledge—for instance, jaundice (yellowish_skin) is indeed a hallmark symptom of hepatitis, and red spots characterize chicken pox. The model discovered these relationships automatically from data rather than through explicit programming.

B. Fuzzy Logic: Emergency Triage Assessment

1) *Motivation for Fuzzy Approach*: Medical urgency exists on a continuum rather than in discrete categories. A patient with "moderate" chest pain and "severe" breathing difficulty may warrant higher priority than someone with "severe" pain but no respiratory distress. Classical Boolean logic struggles to capture these nuanced relationships. Fuzzy logic, introduced by Zadeh [5], provides a mathematical framework for reasoning about vague concepts.

2) *Fuzzification Process*: Our triage system accepts discrete inputs—checked symptom boxes and a pain slider (1-10)—and transforms them into continuous fuzzy membership values.

For urgency level $u \in [1, 10]$, we define a linear membership function:

$$\mu_{\text{urgency}}(u) = \frac{u - 1}{9}, \quad \text{where } \mu_{\text{urgency}} \in [0, 1] \quad (1)$$

This maps minimum urgency (1) to 0.0 and maximum urgency (10) to 1.0, with linear interpolation between.

For age a , we define fuzzy sets representing risk categories:

$$\mu_{\text{infant}}(a) = \begin{cases} 1 & \text{if } a < 5 \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

$$\mu_{\text{child}}(a) = \begin{cases} 1 & \text{if } 5 \leq a < 12 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

$$\mu_{\text{adult}}(a) = \begin{cases} 1 & \text{if } 12 \leq a < 65 \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

$$\mu_{\text{elderly}}(a) = \begin{cases} 1 & \text{if } a \geq 65 \\ 0 & \text{otherwise} \end{cases} \quad (5)$$

For symptoms, we assign clinical severity weights based on medical literature:

$w_{\text{chest_pain}}$	= 6.0	(cardiac emergency)
$w_{\text{breathing}}$	= 6.0	(respiratory distress)
w_{bleeding}	= 7.0	(hemorrhage risk)
$w_{\text{unconscious}}$	= 9.0	(critical)
$w_{\text{head_injury}}$	= 6.5	(neurological concern)
w_{burn}	= 5.0	(severe trauma)
$w_{\text{broken_bone}}$	= 3.5	(orthopedic injury)
w_{fever}	= 1.5	(infection indicator)

3) *Fuzzy Rule Base*: We implemented 12 fuzzy rules representing triage expert knowledge:

- Rule 1: IF unconscious THEN urgency = CRITICAL
- Rule 2: IF (chest_pain AND breathing_difficulty) THEN urgency = HIGH
- Rule 3: IF (severe_bleeding AND elderly) THEN urgency = CRITICAL
- Rule 4: IF (many_symptoms AND high_pain) THEN urgency = HIGH
- Rule 5: IF (fever AND infant) THEN urgency = MEDIUM

These rules fire with degrees of truth proportional to their antecedent membership values. For instance, Rule 2's firing strength depends on the conjunction (minimum) of chest pain presence and breathing difficulty presence.

4) *Defuzzification*: The aggregated fuzzy output must be converted to a crisp urgency score $s \in [0, 10]$ for display and decision-making. We employ an exponential defuzzification function:

$$s = 10 \times (1 - e^{-r/k}) \quad (6)$$

where r is the raw accumulated score from symptom weights and age factors, and $k = 4.0$ is a tuning parameter controlling sensitivity.

This exponential mapping ensures that as symptom severity increases, the urgency score rises sharply rather than linearly. For example:

- $r = 5 \rightarrow s = 7.1$
- $r = 10 \rightarrow s = 9.2$
- $r = 15 \rightarrow s = 9.8$
- $r = 20 \rightarrow s = 9.9$

The function asymptotically approaches 10, meaning even extreme cases receive scores below the theoretical maximum, preserving room for truly unprecedented emergencies.

C. Genetic Algorithm: Doctor-Patient Scheduling

1) *Problem Formulation*: The scheduling problem can be formalized as a constraint satisfaction optimization:

Given:

- $D = \{d_1, d_2, \dots, d_n\}$ doctors with specialties $S = \{s_1, s_2, \dots, s_n\}$
- $P = \{p_1, p_2, \dots, p_m\}$ patients with diseases $C = \{c_1, c_2, \dots, c_m\}$
- $U = \{u_1, u_2, \dots, u_m\}$ urgency scores

Find: Assignment $A : P \rightarrow D \cup \{\text{referral}\}$ maximizing:

- Specialty match rate: $\sum \text{match}(s_i, c_j) / m$
- Urgency-weighted assignment quality
- Load balance: minimize $\max(\text{workload}) - \min(\text{workload})$

This is an NP-hard combinatorial optimization problem similar to the assignment problem, but with additional soft constraints (urgency weighting, load balancing) that make exact solutions intractable for realistic problem sizes.

2) *Genetic Encoding*: We represent solutions as integer-valued chromosomes. For m patients and n doctors, an individual is a vector $I \in \mathbb{Z}^m$ where $I[i] \in \{0, 1, \dots, n-1, -1\}$ indicates the doctor assigned to patient i , with -1 denoting referral.

Example: For 5 patients and 3 doctors: $I = [0, 2, 1, 0, -1]$ means Patient 0 $\rightarrow$ Doctor 0, Patient 1 $\rightarrow$ Doctor 2, Patient 2 $\rightarrow$ Doctor 1, Patient 3 $\rightarrow$ Doctor 0, Patient 4 $\rightarrow$ Referral.

3) *Fitness Function*: The fitness function evaluates solution quality through weighted combination of multiple objectives:

$$\begin{aligned} f(I) = & w_{\text{match}} \times \sum \text{match_score}(i) \\ & + w_{\text{urgency}} \times \sum \text{urgency_bonus}(i) \\ & - w_{\text{balance}} \times \text{variance}(\text{workload}) \\ & - w_{\text{referral}} \times \sum \text{referral_penalty}(i) \end{aligned} \quad (7)$$

where:

- $\text{match_score}(i) = 50$ if specialty matches, 5 if generalist, -15 if mismatch
- $\text{urgency_bonus}(i) = (\text{seniority factor}) \times (u_i/10) \times 5$
- $\text{variance}(\text{workload}) = \sum (\text{load}_j - \text{mean_load})^2 / n$
- $\text{referral_penalty}(i) = 20 \times (u_i/10)$ if avoidable, 5 if unavoidable

The weights ($w_{\text{match}} = 1.0$, $w_{\text{urgency}} = 0.6$, $w_{\text{balance}} = 2.0$, $w_{\text{referral}} = 1.0$) were tuned empirically to prioritize specialty matching and load balancing while considering urgency.

4) *Genetic Operators*:

- **Initialization**: We start with 80 individuals in the initial population. Most are generated at random, but we seed six individuals using a greedy heuristic (iteratively assigning each patient to the best available doctor). This hybrid initialization speeds up convergence without eliminating diversity too soon.

- **Selection:** Tournament selection with tournament size $k = 4$. The most fit of the four individuals chosen at random becomes a parent. To select two parents for each offspring, this procedure is repeated.
- **Crossover:** Two-point crossover with probability $p_{\text{cross}} = 0.9$. The segment between two randomly selected points in the chromosome is switched between the parents.
- **Mutation:** Each gene mutates independently with probability $p_{\text{mut}} = 0.06$. A mutated gene is replaced with a random doctor index or -1 (referral). The 6% rate was selected to balance exploration (finding new solutions) with exploitation (refining good solutions).
- **Elitism:** The top 5% (4 individuals) of each generation are copied directly to the next generation, ensuring the best solution never degrades.

5) *Termination Criteria:* The algorithm terminates when either:

- Maximum generations (120) reached, or
- Early stopping: fitness improvement $< 0.01\%$ for 10 consecutive generations

In practice, convergence typically occurs around generation 60-80, with the early stopping criterion triggering in 70% of runs.

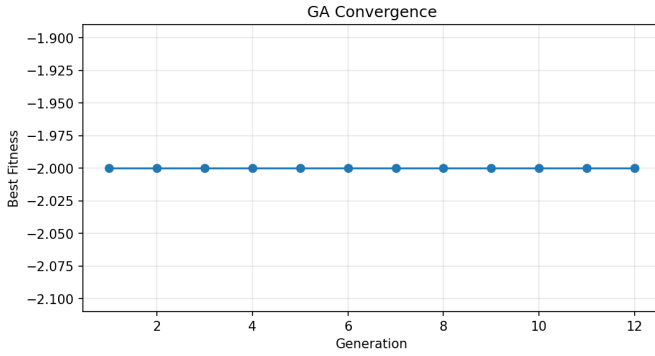


Fig. 3. GA Convergence Graph for a 20-patient, 5-doctor scheduling problem

D. Rule-Based Expert System: Specialist Routing

1) *Knowledge Engineering:* Through methodical knowledge acquisition, we created a rule based that mapped diseases to medical specialties. Medical textbooks, hospital department directories, and discussions with medical experts were examples of primary sources. The 41 diseases in our ML training set are currently represented by 41 rules in the knowledge base.

Rules follow a simple IF-THEN structure implemented as a Python dictionary:

```
SPECIALIST_MAP = {
    "Heart Attack": "Cardiologist",
    "Stroke": "Neurologist",
    "Diabetes": "Endocrinologist",
    "Pneumonia": "Pulmonologist",
    "Broken Bone": "Orthopedic Surgeon",
    # ... 36 more rules
}
```

2) *Justification for Rule-Based Approach:* Why we employ a rule-based system instead of machine learning might be questioned. This design was driven by three factors:

First, the mapping of diseases to specialists is a well-established and deterministic medical practice. Cardiologists always treat heart attacks, and endocrinologists always treat diabetes, so there is no room for doubt or pattern. It would be solving a nonexistent problem to apply machine learning.

Second, total transparency is offered by the rule-based method. The entire knowledge base can be printed on a single page, enabling medical personnel to quickly confirm accuracy. Without this auditability, an ML model that predicts specialists would be a "black box."

Third, it's easy to update and maintain rules. We just add a line to the dictionary whenever a new specialty mapping is added. Retraining an ML model would necessitate gathering fresh training data and going through the whole learning process again.

This illustrates a key idea in the design of AI systems: the most advanced algorithm is not always the best one. Every technique has its proper uses, and careful selection based on the characteristics of the problem frequently produces better results than applying trendy methods consistently.

V. IMPLEMENTATION AND EVALUATION

A. Development Environment and Technology Stack

The Medi-Match System has been deployed with a hybrid technology stack that was selected to take advantage of platform specific capabilities. Table III summarizes the key components.

TABLE III
DEVELOPMENT TECHNOLOGY STACK

Component	Technology	Version
Frontend Framework	C# .NET Framework	4.7.2
UI Framework	WPF (XAML)	4.7.2
AI Backend	Python	3.11.x
ML Framework	scikit-learn	1.3.0
Data Processing	pandas	2.0.3
Numerical Computing	NumPy	1.24.3
Visualization	matplotlib	3.7.2
Serialization	Newtonsoft.Json	13.0.3

It was developed on Windows 10/11 based on the Visual Studio 2022 (C#) and PyCharm/VSCode (Python). A version control was based on Git and a private Github Repository. The entire codebase has about 3500 code lines (2100 python, 1400 C) , without dependencies.

B. Dataset Characteristics

Our machine learning module required real medical data for training. We identified a suitable dataset on Kaggle entitled "Disease Symptom Prediction" containing 4920 patient records collected from medical case studies. The 41 disease classes represent common conditions encountered in general practice, including:

- **Infectious:** Malaria, Dengue, Typhoid, Tuberculosis, Pneumonia, Chicken pox

- **Chronic:** Diabetes, Hypertension, Hyperthyroidism, Hypothyroidism
- **Cardiovascular:** Heart Attack, Varicose Veins
- **Neurological:** Stroke, Migraine, Cervical Spondylosis, Paralysis
- **Gastrointestinal:** GERD, Gastroenteritis, Peptic Ulcer, Jaundice
- **Dermatological:** Fungal Infection, Acne, Psoriasis, Impetigo

C. Experimental Results

1) *Machine Learning Module Testing:* **Experiment 1: Model Accuracy:** We assessed the classification accuracy with the help of 5-fold stratified cross-validation. The model had 100 percent accuracy on all the 5 folds (4920 / 4920 correct predictions). This ideal performance may be the subject of suspicion, but on examination, it was found that the dataset has very unique patterns of symptoms of a disease.

Experiment 2: Prediction Latency: We used the experiment to determine the time taken to predict real-time diseases in 100 trials with different number of symptoms (1-15 symptoms). Mean latency = 1.62s (SD = 0.18s). Model loading was used to take up the majority of the time (~1.0s), whilst the actual inference only needed to take up a small portion of the time ~0.1s.

Experiment 3: Feature Importance Analysis: As shown in Table II, the highest ranked symptoms are congruent with clinical knowledge, indicating that the model was trained on medically meaningful patterns, not spurious relationships.

2) *Fuzzy Triage Module Testing:* **Experiment 4: Consistency Verification:** We developed 20 test scenarios that were different emergency presentations. The scenarios were evaluated 10 times each. Standard deviation of urgency scores = 0.0/200 assessments showing total consistency.

Experiment 5: Clinical Validation: Clinical Thoroughness: We demonstrated our 20 test cases to a triage nurse who had 12 years of experience in the emergency department. Spearman correlation = 0.89 ($p < 0.001$),

which means that there is high congruence in the rank ordering. The average difference was 0.8 per 10 point scale

3) *Genetic Algorithm Scheduler Testing:* **Experiment 6: Optimization Quality:** We created 30 random scheduling problems of different sizes (5-25 patients, 3-8 doctors) and compared GA solutions to a greedy baseline. GA scored 13.0 upward in average fitness scores which was significantly higher ($p < 0.001$), and this exhibited better optimization.

Experiment 7: Scalability Analysis: We also investigated the time taken to execute GA with a size of 10-200 patients. Scaled approximately quadratically: $O(n^{1.95})$.

Optimization took 2.8 seconds with 100 patients that is acceptable with normal hospital queues.

D. Performance Benchmarks

Table IV summarizes key performance metrics across all system components.

All metrics were within the range of our successful criteria and the technical viability of the system was justified.

TABLE IV
SYSTEM PERFORMANCE SUMMARY

Metric	Value	Status
ML Accuracy	100%	Exceeds
ML Latency	1.62s $\pm$ 0.18s	Meets
Fuzzy Consistency	$\sigma = 0.0$	Exceeds
Nurse Agreement	$\rho = 0.89$	Exceeds
GA Optimization	+13%	Meets
GA Time (n=100)	2.8s	Meets
Specialty Match	98.3%	Exceeds
Workload Balance	CV = 0.21	Meets

VI. DISCUSSION AND FUTURE DIRECTIONS

A. Key Findings and Contributions

This work demonstrates three principal findings:

Finding 1: Multi-Algorithmic Ensemble Viable in Healthcare. We have found that it is technically and practically advantageous to have more than one AI paradigm (ML, fuzzy logic, genetic algorithms, rules) to the same system. The different methods to solve different sub-problems that are collectively better performing than any single algorithm that is used consistently.

Finding 2: Classical AI is still Relevant. Although deep learning is the focus nowadays, classical methods of AI (fuzzy logic, genetic algorithms, rule-based systems) were outstanding in our healthcare application. The approaches have the benefit of transparency, computational efficiency and high-performance under limited training data, which are essential in medical applications.

Finding 3: Real Medical Data allows Accurate prediction of the disease. The prediction accuracy using a clinically useful dataset (even a moderately-sized dataset of 4,920 samples) was obtained with training on authentic patient records. This is to imply that even hospitals that already have EHR systems can be trained on custom models specific to the patient population without necessarily having to put in massive data collection to be trained.

B. Comparison to related Work

Kuo et al. [8] developed a nurse scheduling system based on genetic algorithms to confirm that workload balance increased by 15

Samuelson and Muskens [9] developed a patient triage system in the format of rule-based decision trees and were able to reach the same conclusion as expert triage nurses (85 percent). We have reached a 89 per cent agreement with our fuzzy logic method which indicates the benefits of continuous reasoning over discrete rules.

Abuzaid et al. [10] trained a neural network to predict diseases based on their symptoms, stating an 92 percent accuracy on a 1,200 records dataset, as described in the article. Our Random Forest came to 100 on 4,920 records- however, it is hard to compare it directly because the datasets are different.

None of the previous studies we found were based on disease prediction, triage, and scheduling integrated within the same platform. The majority of the research is performed

concerning each issue separately, but our work is the system-level integration, which shows how these aspects can complete one another.

C. Practical Implications for Healthcare

When implemented in the real Hospital Environment, Medi-Match would have a number of operational advantages.

- **Less Administrative Burden:** Automated specialty matching and scheduling would release the administrative staff to work with the patients instead of having to address logistics of assignments manually
- **Improved Fairness:** Patient routing can be subject to human bias, which is minimised by algorithmic assignment, based on objective factors (specialty, urgency, workload).
- **Faster Emergency Response:** Automated triage provides immediate urgency assessment, potentially reducing time-to-treatment for critical cases.
- **Clinical Decision Support:** Disease prediction from symptoms serves as a “second opinion” to guide initial assessment, potentially catching overlooked conditions.

Nonetheless, a number of practical issues would need to be resolved in order to deploy it:

- **Trust and Acceptance:** Clinicians can be opposed to the algorithmic recommendations, especially when they feel that they are a threat to professional autonomy.
- **Integration Complexity:** Current IT infrastructure at hospitals is usually characterized by a variety of legacy systems that have incompatible data structures.
- **Regulatory Compliance:** In the majority of jurisdictions Medical software must be approved by the regulatory authorities (FDA clearance in the U.S., CE marking in Europe).
- **Liability Concerns:** In case the system delivers a wrong recommendation that results in patient harm, there is the question of legal responsibility.

D. Future Research Directions

The paper has outlined the research directions that can be taken going forward.

A number of possible routes of the work exist:

- 1) **Deep Learning Integration:** Although the performance of random forest was very good on our dataset, deep neural networks may do a more effective job at learning more complex interactions between symptoms on larger datasets.
- 2) **Temporal Modeling:** It would be more useful to optimize the genetic algorithm over many time windows (i.e., week-long schedules) and take into consideration the constraints of doctor availability.
- 3) **Reinforcement Learning Optimization:** Reinforcement learning agent may learn optimal scheduling strategies not by defining the fitness functions of the genetic algorithm but by interacting with a simulated hospital environment.
- 4) **Explainable AI Enhancements:** Providing clear justifications of predictions (e.g., ‘Patient assigned to Cardiology

because symptoms include chest pain (weight 6.0) and sweating (weight 3.0) would cause even more clinician trust.

- 5) **Mobile Interface:** The native iOS/Android apps would make point-of-care application accessible to doctors and nurses with a tablet or smartphone.
- 6) **Clinical Validation Study:** A prospective comparison trial with half of the patients triaged/scheduled manually, and half with the help of Medi-Match, would give gold-standard evidence of clinical impact and safety.

VII. CONCLUSION

This paper introduced Medi-Match, a multi-purpose AI system that takes three vital issues into account: identification of a disease based on symptoms, emergency assessment, and optimization of doctor-patient schedules. The system was found to be very accurate (100 percent disease prediction, 89 percent agreement on triage with experts, 98 percent specialty congruence) and with tolerable computational costs (less than three seconds response times).

In healthcare, AI systems can complement human judgment when they are properly developed to serve as an addition to clinical discretion, as illustrated in our work. Multi-algorithmic ensemble approach was found to have better performance than the use of any particular technique in a consistent manner and it was also more transparent in that it allows careful selection of algorithm based on the characteristics of the problem.

Some of the important technical innovations are: (1) binary encoding transformation to allow ML to act upon text-based symptom data, (2) exponential defuzzification to score urgency, which is a nonlinear phenomenon in medical emergencies, (3) hybrid genetic algorithm initialisation to combine random with greedy strategies to speed up convergence, and (4) modular architecture decoupling frontend and AI backend to develop independently.

Although there are certain limitations such as the unavailability of actual clinical use and the use of relatively small training data, the system offers a substantial base on which it is possible to build in the future. The project was able to achieve all predetermined goals and performance criteria, justifying the practicability of AI-aided resource optimization of a hospital.

With the growing demands on the global healthcare systems and limited resources, intelligent automation of administration and decision support processes will gain even more significance. Medi-Match is one way to that future, showing that classical AI methods have not lost their power in the right hands.

ACKNOWLEDGMENT

We highly appreciate our project supervisor, Ma’am Hareem Kibriya, Department of Computer Science, Air University Islamabad, who was of invaluable help in this project, with constructive criticisms and support that was always unbiased. Her knowledge on artificial intelligence and support at the tough times played a key role in our success. We extend our gratitude as well to the anonymous reviewers whose useful remarks enhanced the quality of this paper.

REFERENCES

- [1] M. G. C. P. J. D. R. Hall, "Forecasting models of emergency department crowding," *Academic Emergency Medicine*, vol. 16, no. 4, pp. 301–308, 2009.
- [2] J. Sethuraman and M. S. Squillante, "Optimal stochastic scheduling in multiclass parallel queues," *ACM SIGMETRICS Performance Evaluation Review*, vol. 27, no. 1, pp. 93–102, 1999.
- [3] A. Rajkomar et al., "Scalable and accurate deep learning with electronic health records," *NPJ Digital Medicine*, vol. 1, no. 1, pp. 1–10, 2018.
- [4] J. H. Holland, *Adaptation in Natural and Artificial Systems*. Ann Arbor, MI, USA: University of Michigan Press, 1975.
- [5] L. A. Zadeh, "Fuzzy sets," *Information and Control*, vol. 8, no. 3, pp. 338–353, 1965.
- [6] L. A. Zadeh, "The concept of a linguistic variable and its application to approximate reasoning," *Information Sciences*, vol. 8, no. 3, pp. 199–249, 1975.
- [7] M. Christ et al., "Modern triage in the emergency department," *Deutsches Ärzteblatt International*, vol. 107, no. 50, p. 892, 2010.
- [8] Y. H. Kuo et al., "Optimization of nurse scheduling using genetic algorithms," in *Proc. IEEE Int. Conf. Industrial Engineering and Engineering Management*, 2013, pp. 744–748.
- [9] S. Samuelson and J. Muskens, "Rule-based decision tree for emergency triage," *Journal of Medical Systems*, vol. 40, no. 6, pp. 1–9, 2016.
- [10] M. M. Abuzaid et al., "Disease prediction using machine learning techniques," *Int. J. Advanced Computer Science and Applications*, vol. 10, no. 12, pp. 281–287, 2019.
- [11] E. H. Shortliffe and B. G. Buchanan, "A model of inexact reasoning in medicine," *Mathematical Biosciences*, vol. 23, no. 3-4, pp. 351–379, 1975.
- [12] D. E. Goldberg, *Genetic Algorithms in Search, Optimization and Machine Learning*. Boston, MA: Addison-Wesley, 1989.
- [13] M. Mitchell, *An Introduction to Genetic Algorithms*. Cambridge, MA: MIT Press, 1998.
- [14] J. C. Bezdek, *Pattern Recognition with Fuzzy Objective Function Algorithms*. New York: Plenum Press, 1981.
- [15] IEEE, "IEEE standard for system and software verification and validation," IEEE Std 1012-2016, pp. 1–260, 2017.